FASE 1: INTEGRADOR DE FLUJOS (0-6 MESES)

Rol funcional: No-code/Low-code Automation Specialist con base técnica **Objetivo:** Primeros €1.5-2.5k/mes + fundamentos sólidos **Identidad profesional:** "Ayudo a empresas a automatizar procesos con IA y sin código"

📊 DISTRIBUCIÓN DE TIEMPO SEMANAL

Total: 20-25 horas/semana (ajustable según tu disponibilidad)

Actividad	Horas/semana	%
SQL (PRIORIDAD MÁXIMA)	6-8h	30%
No-code (Make→n8n)	5-6h	25%
Python automatización	4-5h	20%
Proyectos reales	3-4h	15%
Compliance básico	1-2h	5%
Networking/Ventas	1-2h	5%

🎓 COMPETENCIAS TÉCNICAS MÍNIMAS

1. SQL (40% esfuerzo técnico) - PRIORIDAD #1

Por qué primero:

- Te enseña cómo se estructura información empresarial
- Fundamento de TODO sistema de datos
- Arquitectos que no saben SQL = consultores PowerPoint

Nivel objetivo: SQL Intermedio

SQL

✅ SELECT, WHERE, ORDER BY, LIMIT

✅ JOINs (INNER, LEFT, RIGHT, FULL)

✅ GROUP BY, HAVING, COUNT, SUM, AVG

✅ Subqueries

✅ CASE WHEN (lógica condicional)

✅ CTEs (Common Table Expressions) básico

✅ Índices (qué son, cuándo usarlos)

✅ PRIMARY KEY, FOREIGN KEY

✅ Normalización (1NF, 2NF, 3NF conceptual)

✅ Transactions (BEGIN, COMMIT, ROLLBACK)

****Recursos específicos:****

Semana 1-2: Fundamentos

- Curso: "SQL for Data Analysis" (Mode Analytics - gratis)

- Práctica: SQLZoo (ejercicios interactivos)

- Instala: PostgreSQL local + pgAdmin

Semana 3-4: Intermedio

- Curso: "SQL Joins" (HackerRank)

- Práctica: LeetCode SQL (easy → medium)

- Proyecto: Base de datos con 5 tablas relacionadas

Semana 5-6: Aplicado

- Practica queries sobre datos reales

- Dataset: Kaggle (elige uno interesante para ti)

- Objetivo: responder 10 preguntas de negocio con SQL

Criterio de dominio:

- ✅ Puedes escribir JOIN de 3+ tablas sin googlear

- ✅ Entiendes cuándo usar subquery vs JOIN

- ✅ Puedes explicar normalización a no-técnico

- ✅ Identificas problemas de performance (falta de índice)

---

2. NO-CODE/LOW-CODE (30% esfuerzo)

Estrategia de transición: Make → n8n

MES 1-2: Make (Integromat)

Por qué empezar con Make:

- Curva de aprendizaje suave

- Interfaz visual excelente

- Monetización rápida

Competencias:

```
```

✅ Scenarios (flujos)

✅ Triggers (webhooks, schedule, watch)

✅ Routers (lógica condicional)

✅ Iterators (procesar arrays)

✅ Data stores básico

✅ Error handling

✅ API connections (OAuth, API key)

```

**Recursos:**

- Make Academy (gratis)

- YouTube: "Make automation tutorials"

- Práctica: automatiza TUS procesos primero

**Proyectos Make:**

1. Email → Airtable → Slack notification

2. Form submission → Process data → Save to DB

3. RSS feed → Filter → Send digest email

---

**MES 3-4: Transición a n8n**

**Por qué migrar a n8n:**

- Self-hosting (control total)
```

- Ves el JSON (aprendes estructura de datos)

- Puedes inyectar código Python/JavaScript

- Gratis (self-hosted)

- Mentalidad de ingeniero

Competencias n8n:

```

✅ Instalación Docker (n8n self-hosted)

✅ Workflows con código custom

✅ Ver/editar JSON directamente

✅ Credentials management

✅ Webhooks avanzados

✅ Error workflows

✅ Environment variables

```

Recursos:

- n8n Documentation (excelente)

- n8n YouTube channel

- n8n Community (forum)

Proyecto de migración:

Toma tus 3 automatizaciones de Make → recréalas en n8n

- Documenta diferencias

- ¿Qué es más fácil? ¿Qué es más difícil?

- ¿Cuándo usarías cada uno?

MES 5-6: n8n Avanzado + IA Integration

Competencias:

```

✅ Integración con Claude/OpenAI APIs

✅ Prompt chains (multi-step reasoning)

✅ Data transformation con código

✅ Conditional routing complejo

✅ Loop workflows

✅ Sub-workflows (modularidad)
```

Proyectos avanzados:

1. AI content pipeline: input → research → generate → review → publish
2. Customer support: email → classify → route → AI draft → human review
3. Data enrichment: scrape → clean → enrich with AI → save

3. PYTHON PARA AUTOMATIZACIÓN (20% esfuerzo)

NO estás aprendiendo Python para ser developer. Estás aprendiendo Python para:

- Procesar datos que no-code no puede
- Crear glue code entre sistemas
- Entender código de sistemas complejos

Nivel objetivo: Python Funcional (no Experto)

python

✅ Variables, tipos básicos (str, int, float, bool, list, dict)

✅ Condicionales (if/elif/else)

✅ Loops (for, while)

✅ Funciones (def, return, parameters)

✅ List comprehensions básicas

✅ Manejo de archivos (open, read, write)

✅ JSON (json.loads, json.dumps)

✅ CSV (csv.reader, csv.writer)

✅ Requests library (GET, POST, headers, auth)

✅ Try/except (error handling básico)

✅ Datetime (manipulación de fechas)

✅ OS/pathlib (manejo de paths)

✅ Logging básico

```

**NO necesitas (todavía):**

- ❌ POO avanzado

- ❌ Decorators

- ❌ Metaclases

- ❌ Async/await (todavía)

- ❌ Testing exhaustivo

**Recursos:**

**Semana 1-2: Fundamentos**

- "Python Crash Course" (primeros 6 capítulos)

- O: "Automate the Boring Stuff with Python" (caps 1-7)
```

- Practica: 30 min/día en Exercism.io

Semana 3-4: APIs y Datos

- Requests library tutorial
- Working with JSON in Python
- Reading/writing CSV files
- Proyecto: Script que consume API y guarda en CSV

Semana 5-6: Automatización

- "Automate the Boring Stuff" (caps 8-12)
- Proyectos: scripts de automatización real
- Integra con n8n via webhooks

Criterio de dominio:

- ✅ Puedes leer código Python intermedio y entenderlo
- ✅ Puedes modificar script existente sin romperlo
- ✅ Puedes escribir script de 50-100 líneas funcional
- ✅ Entiendes mensajes de error y puedes debuggear

---

*### **4. CONCEPTOS DE SISTEMAS (10% esfuerzo)***

Aprendizaje conceptual, no profundo:

```
```

✅ Qué es una API REST

✅ HTTP methods (GET, POST, PUT, DELETE, PATCH)

✅ HTTP status codes (200, 400, 401, 404, 500, 503)

✅ Autenticación básica (API keys, Bearer tokens, OAuth2 conceptual)

✅ Rate limiting (qué es, por qué existe)

✅ Webhooks vs Polling

✅ JSON estructura

✅ Idempotencia (conceptual)

✅ Qué es servidor, cliente, IP, DNS

✅ HTTPS vs HTTP

```

**Recursos:**

- "APIs for Beginners" (freeCodeCamp YouTube)

- Postman learning center (requests hands-on)

- Explainshell.com (entender comandos)

---

### **5. CLOUD BÁSICO (5% esfuerzo)**

**Nivel: Conceptual + AWS Free Tier hands-on**

```

✅ Qué es cloud computing

✅ IaaS vs PaaS vs SaaS

✅ Concepto de deploy

✅ AWS Free Tier:

- EC2 (servidor virtual) - lanzar uno básico
- S3 (storage) - subir archivos
- RDS (database) - PostgreSQL managed

✅ Concepto de costs (todo en cloud cuesta)

✅ Logs básicos

```
```

Recursos:

- AWS Free Tier (12 meses gratis)
- "AWS Concepts" (Fireship - YouTube, 10 min)
- Deploy tu primera app en EC2

Proyecto:

- Script Python que corre en EC2
- Lee archivos de S3
- Guarda resultados en RDS
- Logs a CloudWatch

---

6. COMPLIANCE BÁSICO (5% esfuerzo) ⭐ NUEVO

Por qué esto te diferencia:

- Mercado europeo = GDPR obligatorio

- 90% de "AI engineers" ignoran esto = tú no

- Solution Architect sin compliance = no contratable en empresas serias

Nivel: Fundamentos GDPR

```

✅ Qué es Personal Identifiable Information (PII)

  - Nombre, email, teléfono, IP, ubicación, etc.

✅ Principios GDPR:

  - Lawfulness (consentimiento)

  - Purpose limitation (uso específico)

  - Data minimization (solo lo necesario)

  - Storage limitation (no guardar forever)

✅ Derechos del usuario:

  - Derecho al acceso

  - Derecho al olvido (delete)

  - Derecho a la portabilidad

✅ Consecuencias: multas hasta €20M o 4% revenue

✅ Data residency (dónde viven físicamente los datos)

```

Recursos:

- GDPR.eu (official, simple language)

- "GDPR Explained" (YouTube, múltiples videos buenos)

- Checklist GDPR para developers

Ejercicio práctico:

Toma uno de tus proyectos de automatización:

1. Identifica qué datos guardas
2. ¿Cuáles son PII?
3. ¿Tienes consentimiento del usuario?
4. ¿Cuánto tiempo los guardas?
5. ¿Puedes eliminarlos si te lo piden?

Resultado: Tu primer audit GDPR básico

🏗️ PROYECTOS OBLIGATORIOS FASE 1

3 proyectos que demuestran competencia sistémica:

PROYECTO 1: Pipeline de Datos con Complejidad Sistémica

Objetivo: Demostrar que puedes integrar múltiples componentes

Spec:

```
Input: Scraping web O API externa

Process:
```

- Limpieza de datos (Python)
- Normalización SQL
- Enriquecimiento con IA (clasificación, resumen)

Storage: PostgreSQL

Output: Dashboard simple O notificaciones automáticas

Scheduler: n8n (ejecuta cada X horas)

```

**Complejidad mínima: 5+ componentes integrados**

**Ejemplos:**

1. **Monitor de precios:**
   - Scrape 3 e-commerce
   - Detecta cambios
   - Enriquece con IA (¿es buen precio?)
   - Alerta si precio baja 20%+

2. **Agregador de noticias personalizado:**
   - RSS de 5 fuentes
   - IA clasifica y resume
   - Guarda en DB
   - Email digest diario

3. **Monitor de competencia:**
   - Scrape webs competidores
```

- Detecta cambios (productos nuevos, precios)
- Análisis con IA
- Report semanal

Documentación obligatoria:

- Diagrama de flujo (draw.io o similar)
- README: qué hace, por qué, cómo usarlo
- Tu primer ADR: por qué esta arquitectura

PROYECTO 2: Sistema con Fallback y Soberanía

Objetivo: Demostrar que entiendes resiliencia de sistemas

Spec:

```
Sistema que:
1. IA primaria (Claude/GPT) para tarea compleja
2. Si IA falla → Fallback a reglas simples
3. Si reglas no aplican → Escala a humano
4. Logging de qué path se usó
5. Metrics: % resuelto por IA, por reglas, por humano
```

Ejemplo: Sistema de soporte customer:

```
Input: Email de cliente
↓
Claude analiza: ¿puedo responder esto?
↓
SI → Claude genera draft
NO → Busca en FAQ (reglas)
  ↓
  SI → Respuesta automática
  NO → Forward a humano + contexto
```

Aprendizaje:

- Sistemas productivos necesitan fallbacks
- IA no es 100% confiable
- Escalado a humano es feature, no bug

Documentación:

- ADR: por qué 3 niveles (no solo IA)
- Metrics: en 100 queries, ¿cuántas resolvió cada nivel?
- Cost analysis: ¿cuánto cuesta cada nivel?

PROYECTO 3: Migración Make → n8n + Self-hosting

Objetivo: Demostrar control de infraestructura

Spec:

```

1. Toma automatización compleja de Make

2. Migra a n8n self-hosted (Docker)

3. Añade features que Make no podía:

   - Código Python custom

   - Manipulación JSON avanzada

   - Logging detallado

4. Documenta ventajas/desventajas

```

Documentación obligatoria:

- Comparison table: Make vs n8n

- Cost analysis: Make pricing vs self-hosting

- ADR: cuándo usarías cada uno

- Tutorial: cómo instalar n8n con Docker

Este proyecto es tu carta de presentación técnica.

💰 MONETIZACIÓN FASE 1

Objetivo: €1.5-2.5k/mes en mes 6

ESTRATEGIA DE PRECIOS

NO aceptes:

- ❌ €300 por landing page genérica
- ❌ €500 por "automatización simple"
- ❌ Trabajos de €20-30/hora

SÍ busca:

- ✅ Proyectos con 3+ componentes integrados
- ✅ €800-2k por proyecto
- ✅ €60-80/hora (Italia) o €80-120/hora (remoto UE)

SERVICIOS ESPECÍFICOS

Mes 1-3: Servicios básicos

```
"Automatización de procesos con IA y No-code"
```

Ejemplos:

- "Automatizo tu flujo de leads (formulario → CRM → email → Slack)"
- "Sistema de soporte básico con IA (FAQs + escalado)"

- "Pipeline de contenido (research → IA → draft → review)"

Pricing: €800-1.5k por proyecto

Timeline: 1-2 semanas

```

**Mes 4-6: Servicios con más valor**

```

"Integración de sistemas + IA"

Ejemplos:

- "Integro tu CRM con sistema de inventario + notificaciones IA"

- "Pipeline de datos: scraping → análisis IA → dashboard"

- "Sistema de automatización híbrido (no-code + código custom)"

Pricing: €1.5-3k por proyecto

Timeline: 2-4 semanas

```

### **DÓNDE BUSCAR CLIENTES**

**NO Upwork/Fiverr genérico** (competencia brutal, pricing bajo)

**SÍ estas estrategias:**
```

1. LinkedIn local (Italia) - MÁS EFECTIVO

```

Perfil optimizado:

"Ayudo a empresas a automatizar procesos con IA + No-code

| Python | n8n | Integración de Sistemas | GDPR Compliant"


Estrategia:

- Conecta con CEOs/Ops de startups italianas (50-200 empleados)

- Comparte casos de estudio (tus proyectos)

- Ofrece "Audit gratuito de automatización" (30 min)

```


2. Networking local (Rovereto/Trento)

```

- Meetups tech

- Coworking spaces

- Cámara de comercio

- Asociaciones empresariales


Pitch:

"Ayudo a empresas a reducir 40% del tiempo en tareas repetitivas

usando automatización + IA, sin contratar developers"

```


3. Nicho italiano específico

```

Ejemplos de nichos:

- Estudios legales (automatización de docs)

- Agencias marketing (contenido + reportes)

- E-commerce pequeño (inventario + notificaciones)

- Consultoría (reporting automático)


Ventaja: hablas el idioma, entiendes el contexto local

```


4. Warm outreach

```

Contactos directos:

- Ex-compañeros de tu esposa

- Contactos del fútbol

- Red personal


Mensaje:

"Ahora ayudo a empresas a automatizar procesos.

¿Conoces alguna empresa que pase muchas horas en tareas repetitivas?"

```


PROPUESTA DE VALOR CLARA

NO digas: "Hago automatizaciones con Make/n8n"

SÍ di: "Reduzco 10-20 horas/semana de trabajo manual usando IA + automatización, sin que necesites contratar developers"

Template de propuesta:

```

Problema identificado:

[cliente pasa 15h/semana haciendo X manual]


Solución propuesta:

Sistema automatizado que:

- [hace Y automáticamente]

- [detecta Z con IA]

- [notifica cuando requiere atención humana]


Resultado esperado:

- Ahorro: 12-15h/semana

- ROI: recuperas inversión en 2-3 meses

- GDPR compliant (datos seguros)


Inversión: €1.800 (one-time)

Timeline: 3 semanas

Soporte: 1 mes incluido

```

✅ VALIDACIÓN DE FASE 1 → FASE 2

No pases a Fase 2 hasta cumplir TODOS estos criterios:

Validación Técnica:

- [ ] **SQL:** Puedes escribir JOIN de 3+ tablas sin googlear
- [ ] **Python:** Puedes leer y modificar código de 100+ líneas
- [ ] **APIs:** Has integrado 5+ APIs diferentes
- [ ] **n8n:** Tienes 3+ workflows complejos funcionando
- [ ] **Cloud:** Tienes algo desplegado en AWS/similar

Validación de Proyectos:

- [ ] 3 proyectos completos con 4+ componentes cada uno
- [ ] Al menos 1 sistema con fallback/resiliencia
- [ ] 1 migración Make → n8n documentada
- [ ] Portfolio GitHub con READMEs claros

Validación de Negocio:

- [ ] €1.5k+/mes sostenidos durante 3 meses
- [ ] 3-5 clientes satisfechos (o proyectos pagados)
- [ ] Al menos 1 testimonial/referencia

Validación de Mentalidad (crítica):

- [ ] Tienes 3+ ADRs escritos

- [ ] Puedes explicar tus sistemas a no-técnico

- [ ] Puedes dibujar arquitectura sin ver código

- [ ] Has identificado PII en tus sistemas

Validación de Posicionamiento:

- [ ] LinkedIn optimizado con proyectos

- [ ] Te llaman "quien automatiza" (no "programador")

- [ ] Alguien te pidió recomendación técnica

🚨 SEÑALES DE ALERTA FASE 1

Revisa cada mes:

🔴 ALERTA ROJA (cambia estrategia YA):

- Mes 4 sin ingresos tech

- Solo has hecho tutoriales, cero proyectos reales

- Clientes piden cosas fuera de tu alcance constantemente

- Compartes código pero no puedes explicar decisiones

Acción: Vuelve a fundamentos. Enfócate en 1 skill a la vez.

🟡 ALERTA AMARILLA (ajusta rumbo):

- Mes 3 con ingresos <€500

- Proyectos muy simples (2 componentes)

- No documentas decisiones

- Tutorial hell (5+ cursos, poca práctica)

Acción: Prioriza proyectos reales. Menos teoría, más construcción.

🟢 SEÑAL VERDE (vas perfecto):

- Ingresos creciendo mes a mes

- Clientes te piden más trabajo

- Proyectos cada vez más complejos

- Documentas naturalmente

Acción: Mantén ritmo. Prepara transición a Fase 2.

📅 CALENDARIO SEMANAL TIPO - FASE 1

Semana típica (20-25h totales):

LUNES (4-5h)

```

08:00-09:30 → SQL (tutorial/ejercicios)

10:00-12:00 → Trabajo cliente/proyecto actual

12:00-13:00 → Documentar lo aprendido (ADR, notas)
```

```

### **MARTES (4-5h)**

```
08:00-09:00 → Python (ejercicios prácticos)
09:00-11:00 → n8n workflow (desarrollo)
11:00-12:00 → Buscar clientes (LinkedIn, outreach)
```

### **MIÉRCOLES (4-5h)**

```
08:00-10:00 → Proyecto personal (portfolio)
10:00-11:30 → APIs + integración
11:30-12:00 → Compliance (GDPR reading)
```

JUEVES (4-5h)

```
08:00-09:00 → SQL (queries complejos)
09:00-11:00 → Trabajo cliente/freelance
11:00-12:00 → Networking (LinkedIn, eventos)
```

VIERNES (4-5h)

```

08:00-10:00 → Proyecto personal/experimental

10:00-11:00 → Cloud (AWS hands-on)

11:00-12:00 → Review semanal + planificación

12:00-13:00 → IA como copiloto (consultas Claude)

**SÁBADO/DOMINGO:** Descanso o catch-up si necesario